Constructor in java | Constructor chaining, Overloading & this | java full Cource | Day-13

```
Public class Day13
Public static void main (string[] args){
Student s1 = new student ();  // default values.
System.out.Println(s1.name);
System.out.Println (s1.rollnum);
System.out.Println(s1.age);
System.out.Println (s1.college);
}}
```

: Null
: 0
: 0
: Null

s1.name = "sachin"
s1.age = 20;
s1.rollnum = 95;
s1.college = "IES university"

```
class student {
  String name; // information/data/characteristics. → instance variable
  int age;
  int rollnum;
  String college;
  void markattendance () { // behaviour → function → instance Methods.
    System.out.Println ("Attendence marked for student." + name);
  }
}
```

```
int x;  // Local variable. → No default value
System.out.Println (x);
```
[Compiler error]

* Integer → 0
  Floating → 0.0
  Boolean → false
  String → Null → Nothing

constructors → To create an object.

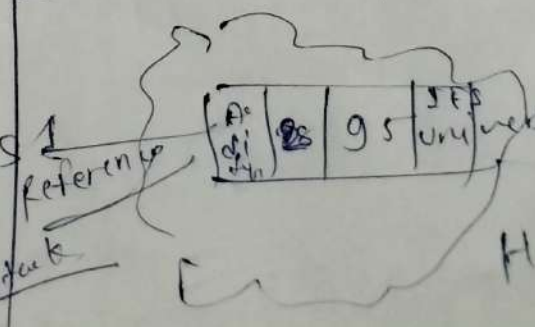
Rules of constructors.

1. Same name as class
2. No return type Not even void
3. Automaticaly called during object creation
4. used to initialize an object
5. It can also be overloaded.

```
class student
  String name;
  int age;
  int roll no;
  String college;
  Student () {  // constructor
    name = "Aditya"
    age = 28;
    rollno = 101;
    college = IES University
```

student s1 = new student ();

data type	Reference	memory Allocation dynamicaly	constructor Call
User define			



```
class student ( ) {
String name;
int age;
int rollno;
String college;

student () {
    empty
}
}
```

Student s1 = new Student ( ) {

Constructor is Mandatory

⇓

(1) Default constructor

default value: print instance. Variable

- Null
- 0
- 0
- Null

Parameterised Constructor

```
class student {
string name;
int age;
int rollno;
string college;
student (string n, int a, int rn, string c) {
    name = n;
    age = a;
    rollno = rn;
    College = c;

student () {
  }   // default Constructor
}
```

student s1 = new Student ("Aditya", 28, 101, "IIT GUWAHATI")

student s2 = new student ();

This keywords.

```
class student {
string name;
int age;
int rollno;
string college;
student (string name, int age, int rollno, string colleg) {
    this.name = name;
    this.age = age;
    this.rollno = rollno;
    this.college = IES University;
```

Prefer variable same name

Java Convention,

~~Unde~~ Easy understanding / Reading

Chaining in Constructor :→ Constructor Overload + Chaining

```
Public class Day 13 {
Public static void main (string [] args) {
Student s1 = new student ();
Student s2 = new student ("Aditya");
Student s3 = new student ("Aditya",28);
Student s4 = new student ("Aditya", 28, 101);
Student s5 = new student ("Aditya", 28, 101, "IES University");
System.out.print ln (s1.name);
System.out.Print ln (s1.age);
System.out.Print ln (s1.roll no);
System.out.Print ln (s1.college);
}}
```

// Constructor calling

this. Current object ke reference ko store karta.
Constructor Call using this. keyword

```
Class student {
String name;
int age;
int rollnum;
String college;

student () {
this ("unknown");
System.out.Print ln ("I am in first Constructor");
}
student (string name) {
this ("~~unknown~~ name., 0);
System.out.Print ln ("I am in 2nd constructor");
}
student (string name., int age) {
this (name, 0, ~~college~~ 0);
System.out.Print ln ("I am in 3rd Constructor");
}
student (string name., int age., int rollnum) {
this (name., 0, 0, "unknown");
System.out.Print ln ("I am in 4rth constructors);
}
student (string name., int age., int roll num, string College) {
~~this(name, 0, 0, "IES University)~~;
this.name = name.;
this.age = age;
this.rollnum = rollnum;
this.College = College
System.out.Print ln ("I am in 5th Constructor");
```

Constructor → manually call.

→ X | No |

student s1 = new Student();

this(name, age); dynamically. Run time.

Student (— — —) X.

No space in Heap.

• Run time exception.

Not enough space in Heap

Heap full